# [TICV2026] Rendu 1 : Calibration de caméra et estimation de pose 3D

Master 2 VMI – Séquences Vidéo | Nguyen Ho Bao Khanh | Novembre 2026

## 1 Introduction et rappels théoriques

L'objectif de ce travail pratique est d'étudier expérimentalement un capteur optique passif (la caméra d'un smartphone) afin d'estimer ses paramètres intrinsèques, ses coefficients de distorsion optique et sa pose 3D par rapport à une mire planaire connue.

*Modèle sténopé et coordonnées homogènes.* D'après le modèle projectif présenté dans le cours (`Seq_Video1`), la projection perspective d'un point 3D du repère monde $M_W = (X_W, Y_W, Z_W)^T$ vers un point image 2D $(u, v)^T$ est non linéaire en coordonnées cartésiennes ($x = fX_c/Z_c$, $y = fY_c/Z_c$). Le passage en coordonnées homogènes permet de linéariser cette transformation sous forme matricielle :

$$s \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = M_{3\times4} \begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix} = K \, [R \mid T] \begin{pmatrix} X_W \\ Y_W \\ Z_W \\ 1 \end{pmatrix}, \quad \text{avec } K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \tag{1}$$

où $K$ est la matrice intrinsèque contenant les focales en pixels $(f_x, f_y)$ et le point principal $(c_x, c_y)$, et $[R \mid T] \in SE(3)$ regroupe la matrice de rotation $R_{3\times3}$ et le vecteur de translation $T = (T_x, T_y, T_z)^T$ définissant les paramètres extrinsèques. Pour un motif 3D non coplanaire, le cours présente deux méthodes linéaires par SVD : `PROJ_MAT_CAL` (estimation directe des 11 paramètres indépendants de $M_{3\times4}$ via $Am = 0$ pour $N \geq 6$ points, Slide 97) et `PARAM_CAL` (système $Av = 0$ à 8 inconnues pour $N \geq 7$ points et orthocentre des points de fuite, Slide 90).

*Distorsions optiques et méthode de Zhang (OpenCV).* Les lentilles réelles des caméras de téléphone introduisent des distorsions géométriques, principalement radiales $(k_1, k_2, k_3)$ et tangentielles $(p_1, p_2)$, appliquées aux coordonnées caméra normalisées $(x, y) = (X_c/Z_c, Y_c/Z_c)$ avant projection en pixels ($u = f_x x_{\text{dist}} + c_x$, $v = f_y y_{\text{dist}} + c_y$) :

$$\begin{cases} x_{\text{dist}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2) \\ y_{\text{dist}} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy \end{cases} \qquad \text{où } r^2 = x^2 + y^2. \tag{2}$$

La fonction `cv2.calibrateCamera()` implémente l'algorithme de Zhang : à partir de plusieurs vues d'une mire plane ($Z_W = 0$), chaque vue fournit une homographie $H = K[r_1, r_2, T]$ donnant deux contraintes linéaires sur $K^{-T}K^{-1}$, suivies d'une optimisation non linéaire de Levenberg-Marquardt minimisant l'erreur de reprojection sur l'ensemble des coins détectés.

*Détermination de la pose (PnP) et article de Quan & Lan (PAMI 1999).* Une fois la matrice $K$ et la distorsion connues, l'estimation de la pose $(R, T)$ à partir de $N$ points 3D et de leurs projections 2D correspond au problème *Perspective-n-Point* (PnP). L'article étudié de Long Quan et Zhongdan Lan (*Linear N-Point Camera Pose Determination*, IEEE TPAMI, 1999) part du fait que les distances inconnues $x_i$ entre le centre optique et chaque point de référence vérifient, pour chaque paire de points, $d_{ij}^2 = x_i^2 + x_j^2 - 2x_i x_j \cos\theta_{ij}$, où $d_{ij}$ est la distance 3D connue entre les points et $\theta_{ij}$ l'angle de vue calculé à partir de $K$. Les auteurs en déduisent des algorithmes linéaires (SVD) en deux étapes pour 4 points et en une étape pour 5 points ou plus, qui donnent une solution unique pour des points génériques et ne dégénèrent pas dans le cas coplanaire (exactement le cas de notre échiquier où $Z_W = 0$) ; la pose $(R, T)$ est ensuite obtenue par orientation absolue. Dans ce TP, nous utilisons `cv2.solvePnP()`, qui résout le même problème PnP mais avec sa méthode par défaut (itérative, de type Levenberg-Marquardt) et non avec l'algorithme de Quan et Lan.

## 2 Protocole expérimental

- **Capteur étudié et métadonnées EXIF :** Nous avons utilisé la caméra grand-angle principale (1×) d'un smartphone Samsung Galaxy S26 Ultra. Les métadonnées EXIF extraites directement des fichiers originaux indiquent : `Model = Galaxy S26 Ultra`, ouverture $f/1.4$ (`FNumber = 1.4`), focale nominale à l'infini `FocalLength = 6.5 mm`, focale équivalente plein format $24 \times 36$ mm `FocalLengthIn35mmFilm = 23 mm`, et résolution native de $4000 \times 2252$ pixels (format 16 : 9). D'après les spécifications publiques du constructeur (non vérifiées par nos soins, l'EXIF ne donnant pas la taille du capteur), cette caméra utilise un capteur CMOS ISOCELL HP2 ($1/1.3''$) en regroupement de pixels (*pixel binning* 16-en-1). Nous faisons l'hypothèse d'un pas de pixel de $2.4\,\mu$m, soit pour une image de $4000 \times 2252$ pixels une surface active de $W_{\text{capteur}} = 9.60$ mm par $H_{\text{capteur}} = 5.405$ mm. La focale exprimée en millimètres dépend directement de cette hypothèse (section 3.2).

- **Mire de calibration :** Échiquier plan imprimé à l'échelle 100% sur feuille A4 comportant $10 \times 7$ cases, soit une grille de $9 \times 6$ coins intérieurs (54 points par image), avec une taille de case de 25.0 mm. La feuille a été fixée au ruban adhésif sur un support rigide plat.
- **Acquisition des 24 vues et code source :** Nous avons acquis 24 images en résolution native ($4000 \times 2252$ pixels) sans zoom (`DigitalZoomRatio = 1.0`). Le script Python (`calibrate.py`) ainsi que l'ensemble des 24 images originales sont disponibles sur le dépôt GitHub : https://github.com/Aplus59/Calibration_Camera. Le protocole d'acquisition suit trois groupes de prises de vue :
  1. *Vues de face étalonnées au mètre ruban (images 1 à 3) :* Téléphone tenu face à la mire à des distances mesurées de 30 cm, 40 cm et 50 cm afin de vérifier la justesse métrique de la profondeur $T_z$.
  2. *Vues inclinées à main levée (images 4 à 19) :* Prises en inclinant librement le téléphone sous différents angles (gauche/droite, haut/bas, diagonales, sans mesurer l'angle) à une distance d'environ 40 cm afin de lever la dégénérescence planaire sur la focale.
  3. *Vues décentrées (images 20 à 24) :* Prises en plaçant la mire vers les quatre coins et les bords du cadre (autour de 50 cm pour les images 20 à 23, et plus près pour l'image 24) afin de contraindre la distorsion en périphérie.



(a) Image 1 : vue frontale (30 cm) (b) Image 8 : vue inclinée (c) Image corrigée (`undistort`)

Figure 1: Détection des $9 \times 6$ coins (`findChessboardCorners`, `cornerSubPix`) et correction de distorsion (`undistort`, $\alpha = 1$).

# 3 Résultats de la calibration

## 3.1 Paramètres intrinsèques, distorsion et erreur de reprojection

Les 54 coins intérieurs ont été détectés sur les 24/24 images en résolution native ($4000 \times 2252$). La fonction `cv2.calibrateCamera()` fournit la matrice intrinsèque $K$ et le vecteur de distorsion suivants :

$$K = \begin{pmatrix} 2754.69 & 0 & 2010.34 \\ 0 & 2753.11 & 1094.86 \\ 0 & 0 & 1 \end{pmatrix} \text{ (en pixels)}, \quad \text{dist} = \begin{pmatrix} k_1 \\ k_2 \\ p_1 \\ p_2 \\ k_3 \end{pmatrix}^T = \begin{pmatrix} 0.2362 \\ -1.0986 \\ -0.0020 \\ 0.0019 \\ 1.8540 \end{pmatrix}^T$$

- *Cohérence géométrique :* Le centre optique estimé $(c_x, c_y) = (2010.34, 1094.86)$ px est très proche du centre géométrique théorique de l'image $(2000, 1126)$ px. Les focales $f_x = 2754.69$ px et $f_y = 2753.11$ px donnent un rapport d'aspect $f_y/f_x = 0.9994 \approx 1$, confirmant que les pixels sont bien carrés.
- *Analyse des coefficients de distorsion :* Les coefficients tangentiels ($p_1, p_2 \approx 2 \times 10^{-3}$) sont quasi nuls, ce qui indique un bon parallélisme entre la lentille et le capteur. Les termes radiaux ($k_1 = 0.236, k_2 = -1.099, k_3 = 1.854$) sont de valeurs élevées et de signes alternés : les trois coefficients du polynôme d'ordre 6 sont fortement corrélés et se compensent, ils doivent donc être interprétés avec prudence individuellement ; c'est surtout leur effet global qui compte (Figure 1c).
- *Erreur de reprojection :* L'erreur quadratique moyenne (RMS) globale retournée par `calibrateCamera` est de 0.8088 px sur des images de $4000 \times 2252$ pixels (soit un RMSE moyen par image de 0.7905 px, Figure 2). Notons que la formule du tutoriel OpenCV (`cv2.norm(imgpoints[i], imgpoints2, cv2.NORM_L2) / len(imgpoints2)`) divise la norme $L_2$ par $M = 54$ au lieu de $\sqrt{M}$, ce qui donne RMSE$/\sqrt{54} = 0.1076$ px. Dans les deux cas, l'erreur reste inférieure au pixel.

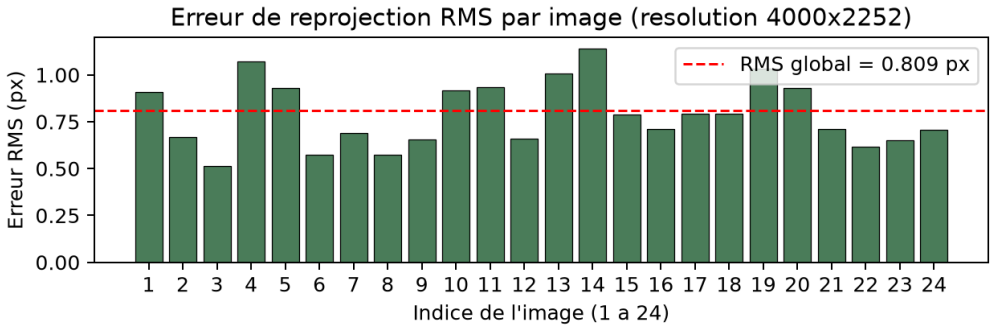



Figure 2: Erreur de reprojection RMS par image sur la résolution native $4000 \times 2252$ (RMS global = 0.809 px).

### 3.2 Focale en mètres via `calibrationMatrixValues()` (PS 3)

Le Tableau 1 présente les paramètres physiques obtenus via `cv2.calibrationMatrixValues()` pour la surface active du capteur ($9.60 \times 5.405$ mm) et les compare aux métadonnées EXIF du téléphone :

| **Paramètre physique** | **Symbole** | **Valeur estimée (OpenCV)** | **Métadonnées EXIF / Capteur** |
|---|---|---|---|
| Focale physique (en millimètres) | $f$ (mm) | 6.6112 mm | `FocalLength` = 6.50 mm (à l'infini) |
| **Focale physique (en mètres)** | **f (m)** | **0.006611** m ($6.61 \times 10^{-3}$ m) | 0.006500 m |
| Champ visuel horizontal / vertical | $\mathrm{FOV}_x, \mathrm{FOV}_y$ | 71.96° / 44.48° | Équiv. 23 mm plein format |
| Point principal sur le capteur | $(c_x, c_y)$ | (4.825 mm, 2.628 mm) | Centre : (4.800 mm, 2.702 mm) |
| Rapport d'aspect des pixels | $f_y/f_x$ | 0.9994 | 1.0000 |

Tableau 1: Résultats physiques fournis par `cv2.calibrationMatrixValues()` et comparaison avec l'EXIF.

*Interprétation physique par la formule des lentilles minces (Slide 15 du cours) :* La focale issue de la calibration vaut $f = 0.006611$ m (6.611 mm) avec l'hypothèse $W_{\text{capteur}} = 9.60$ mm, contre $f_\infty = 6.50$ mm dans l'EXIF (soit +1.71%). Précisons que seule la focale en pixels, $f_x = 2754.69$ px, est mesurée directement : sa conversion en mètres dépend de la largeur du capteur. Avec la largeur pleine du HP2 ($\approx 9.79$ mm) on obtiendrait 6.74 mm ; si l'on suppose exacte la focale EXIF de 6.50 mm, la largeur implicite serait de 9.44 mm. L'incertitude sur $f$ en millimètres est donc de l'ordre de $\pm 2\%$. Un effet physique va dans le même sens : d'après la relation des lentilles minces vue au cours (`Seq_Video1`, Slide 15), lorsque la caméra fait la mise au point (autofocus) sur un objet situé à la distance moyenne $z \approx 400$ mm (40 cm), le tirage optique effectif $z'$ entre la lentille et le capteur devient :

$$\frac{1}{z'} + \frac{1}{z} = \frac{1}{f_\infty} \implies z' = \frac{f_\infty \cdot z}{z - f_\infty} = \frac{6.50 \times 400}{400 - 6.50} = 6.6074 \text{ mm} \quad (+1.65\%)$$

Ce déplacement théorique (environ +1.7%) est du même ordre de grandeur que l'écart observé, mais il reste inférieur à l'incertitude liée à la taille du capteur : nos données ne permettent pas de séparer ces deux effets, et nous pouvons seulement conclure à un accord de l'ordre de 2% avec la focale annoncée.

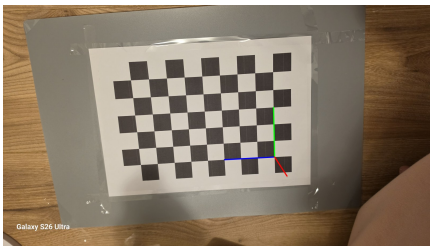
## 4 Aller plus loin : Estimation de pose 3D (`solvePnP`) et expériences

### 4.1 Réalité augmentée : projection d'axes 3D et d'un cube virtuel (`py_pose`)

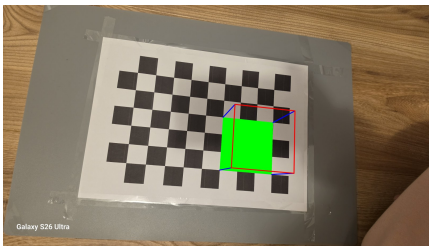
En suivant le tutoriel `py_pose`, nous avons utilisé `cv2.solvePnP(objp, corners2, mtx, dist)` pour calculer les vecteurs de rotation `rvec` et de translation `tvec` de chaque vue, puis `cv2.projectPoints()` pour projeter sur l'image un trièdre 3D de longueur $3 \times 25 = 75$ mm et un cube virtuel de côté 75 mm posé à l'origine de l'échiquier (Figure 3). Le cube reste bien aligné sur la grille quelle que soit l'inclinaison ou l'excentrement de la mire.

### 4.2 Expérience 1 : Vérification métrique des distances caméra–mire (Tz)

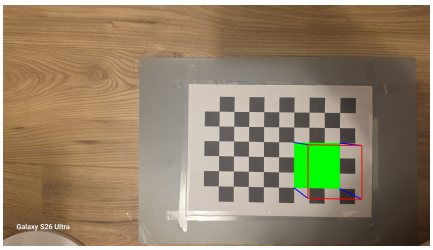
Lors de la prise de vue, nous avons placé le téléphone à 30 cm, 40 cm et 50 cm pour les trois premières vues frontales (distances mesurées au mètre ruban), puis à des distances seulement estimées à l'œil pour les autres vues (environ 40 cm pour les images 4 à 19 et 50 cm pour les images 20 à 23). Seules les trois premières constituent donc une vérification métrique : comme ces vues sont quasi frontales (inclinaison $\leq 1.3°$), la profondeur $T_z$ de l'origine de la grille suivant l'axe optique $Z_c$ correspond directement à la distance orthogonale caméra–mire (Tableau 2) :



(a) Trièdre 3D (image 8)



(b) Cube 3D vue inclinée (image 16)



(c) Cube 3D au bord (image 22)

Figure 3: Estimation de pose 3D avec `solvePnP` : projection des axes $(X, Y, Z)$ et du cube $75 \times 75 \times 75$ mm.

| **Images** | **Configuration du protocole** | **Distance visée** | **Profondeur $T_z$ (`solvePnP`)** | **Écart** |
|---|---|---|---|---|
| Image 1 | Vue de face (proche) | 30.0 cm | 30.6 cm (306.0 mm) | +0.6 cm (2.0%) |
| Image 2 | Vue de face (moyenne) | 40.0 cm | 39.6 cm (396.4 mm) | −0.4 cm (1.0%) |
| Image 3 | Vue de face (éloignée) | 50.0 cm | 49.1 cm (491.3 mm) | −0.9 cm (1.8%) |
| Images 4 à 19 | Vues inclinées à main levée | ≈ 40 cm (estimée) | 40.3 cm à 47.4 cm | – |
| Images 20 à 23 | Vues aux bords / coins du cadre | ≈ 50 cm (estimée) | 49.6 cm à 53.9 cm | – |

Tableau 2: Comparaison entre la distance du protocole de prise de vue et l'estimation $T_z$ par `solvePnP`.

### 4.3 Expérience 2 : Influence du nombre et de la géométrie des vues sur la calibration

Nous avons relancé `calibrateCamera` sur des sous-ensembles croissants des $N$ premières vues, pour $N$ allant de 3 à 24 :

| **Nb d'images $N$** | **N = 3** (front.) | **N = 5** | **N = 8** | **N = 12** | **N = 16** | **N = 20** | **N = 24** |
|---|---|---|---|---|---|---|---|
| Focale $f_x$ (px) | 15803.1 *(dégén.)* | 2774.8 | 2759.3 | 2776.0 | 2777.2 | 2752.0 | 2754.7 |
| Focale $f$ (mm) | 37.93 mm | 6.66 mm | 6.62 mm | 6.66 mm | 6.67 mm | 6.60 mm | 6.61 mm |
| Erreur RMS (px) | 0.678 | 0.690 | 0.716 | 0.739 | 0.787 | 0.831 | 0.809 |

Tableau 3: Convergence de la focale en fonction du nombre d'images $N$ utilisées ($4000 \times 2252$).

Avec $N = 3$ vues purement frontales (images 1 à 3, parallèles au plan image), le système est géométriquement dégénéré : une simple homothétie entre 30, 40 et 50 cm ne permet pas de dissocier la focale $f$ de la profondeur $T_z$, produisant une focale erronée de 37.93 mm malgré un faible RMS (0.678 px). Dès l'ajout des vues inclinées ($N = 5$), la perspective lève cette ambiguïté et la focale converge vers 6.61 mm.

## 5 Discussion : Résolution vs qualité d'image (PS 2) et Conclusion

Comme le soulignent les articles du PS 2 (*Comparer les résolutions 480p à 8K* et *HD contre mégapixels*), un nombre élevé de mégapixels sur un smartphone ne garantit pas à lui seul la qualité optique ni la précision géométrique :

1. *Limites optiques et physiques du capteur :* Sur un capteur compact ($9.6 \times 5.4$ mm), des photosites natifs de $0.6\,\mu$m sont limités par la diffraction et le bruit photonique, ce qui oblige le constructeur à regrouper 16 pixels en 1 ($2.4\,\mu$m, résolution $4000 \times 2252$). De plus, la compacité des lentilles asphériques moulées introduit de la distorsion radiale en périphérie.

2. *Traitement électronique (ISP) et compression :* L'électronique de traitement (lissage du bruit, accentuation des contours, compression JPEG) modifie localement les gradients d'intensité utilisés par `cornerSubPix`. D'ailleurs, lors d'un premier test avec des images compressées via l'application *Phone Link* (réduites autour de $2560 \times 1440$ sans EXIF), la perte de résolution et des métadonnées dégradait l'analyse physique par rapport aux fichiers originaux.

**Conclusion.** Ce TP nous a permis de calibrer la caméra principale de notre smartphone sur ses images natives ($4000 \times 2252$) : nous avons estimé sa matrice intrinsèque $K$ (RMS = 0.809 px), corrigé sa distorsion (`undistort`), déterminé sa focale physique à $f = 0.006611$ m via `calibrationMatrixValues()` (accord de l'ordre de 2% avec l'EXIF $f_\infty = 6.50$ mm, sous l'hypothèse d'un capteur de 9.60 mm de large), et vérifié l'estimation de pose 3D (`solvePnP`) à moins de 1 cm des distances mesurées au mètre sur les trois vues frontales.